WEEK 4 – SQL ORDER MANAGEMENT SYSTEM

SQL Script and Output Screenshots

1. Aim

To create an Order Management Database and perform SQL operations such as table creation, data insertion, updating, deletion, joins, aggregate functions, and subqueries.

2. Database Tables

Database name: OrderDB

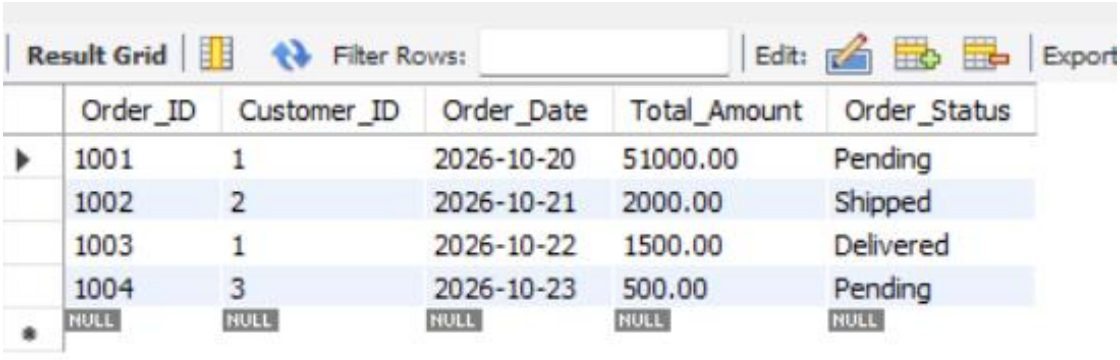
- Customer – stores customer ID, name, email, and city.
- Product – stores product ID, product name, and price.
- Orders – stores customer orders, order date, total amount, and status.
- Order_Details – stores the products, quantities, and prices for each order.

3. SQL Concepts Used

- **Primary key –** Uniquely identifies each row.
- **Foreign key –** Links related tables.
- **NOT NULL and UNIQUE –** Enforce required values and prevent duplicates.
- **CHECK and DEFAULT –** Validate values and provide default values.
- **CRUD –** Create, Read, Update, and Delete database data.
- **JOIN –** Combines related data from multiple tables.
- **Aggregate functions –** Use COUNT, SUM, and AVG to summarize data.
- **Subquery –** Uses a query inside another query.

4. Output Screenshots

The following screenshots show the SQL query results. Each screenshot is placed with a caption for easy identification.



Order_ID	Customer_ID	Order_Date	Total_Amount	Order_Status
1001	1	2026-10-20	51000.00	Pending
1002	2	2026-10-21	2000.00	Shipped
1003	1	2026-10-22	1500.00	Delivered
1004	3	2026-10-23	500.00	Pending
NULL	NULL	NULL	NULL	NULL

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Order_ID	Customer_Name	Product_Name	Quantity	Price	Subtotal	Order_Date	Total_Amount	Order_Status
1001	Aparna	Laptop	1	50000.00	50000.00	2026-10-20	51500.00	Shipped
1001	Aparna	Mouse	3	500.00	1500.00	2026-10-20	51500.00	Shipped
1002	Rahul	Keyboard	2	1000.00	2000.00	2026-10-21	2000.00	Delivered
1003	Aparna	Headphone	1	1500.00	1500.00	2026-10-22	1500.00	Delivered

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Customer_ID	Customer_Name	Order_ID	Order_Date	Product_Name	Quantity	Price	Order_Status
1	Aparna	1001	2026-10-20	Laptop	1	50000.00	Shipped
1	Aparna	1001	2026-10-20	Mouse	3	500.00	Shipped
1	Aparna	1003	2026-10-22	Headphone	1	1500.00	Delivered
2	Rahul	1002	2026-10-21	Keyboard	2	1000.00	Delivered

Result Grid | Filter Rows: | Expor

Product_Name	Order_ID	Quantity	Price
Laptop	1001	1	50000.00
Mouse	1001	3	500.00
Keyboard	1002	2	1000.00
Headphone	1003	1	1500.00

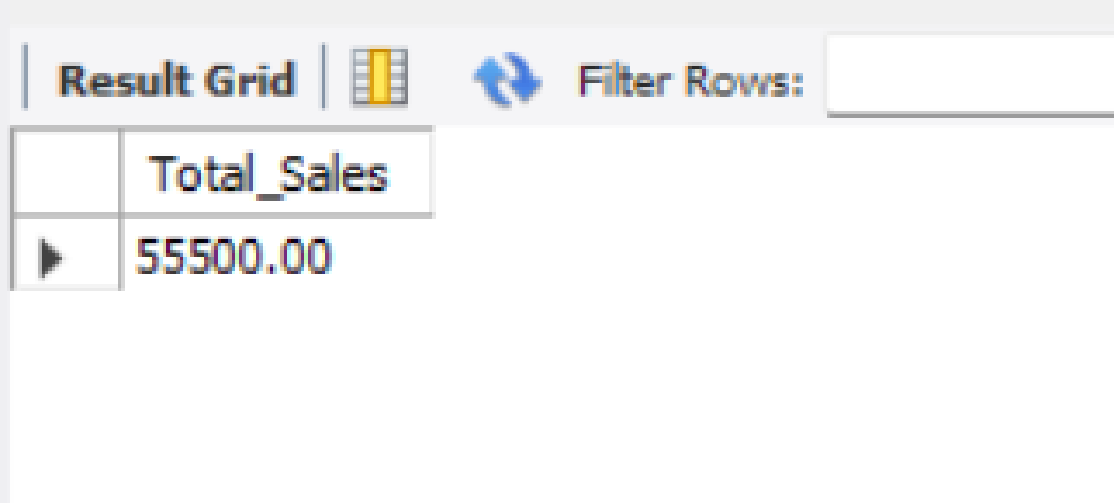
Figure 4: Product-wise order details

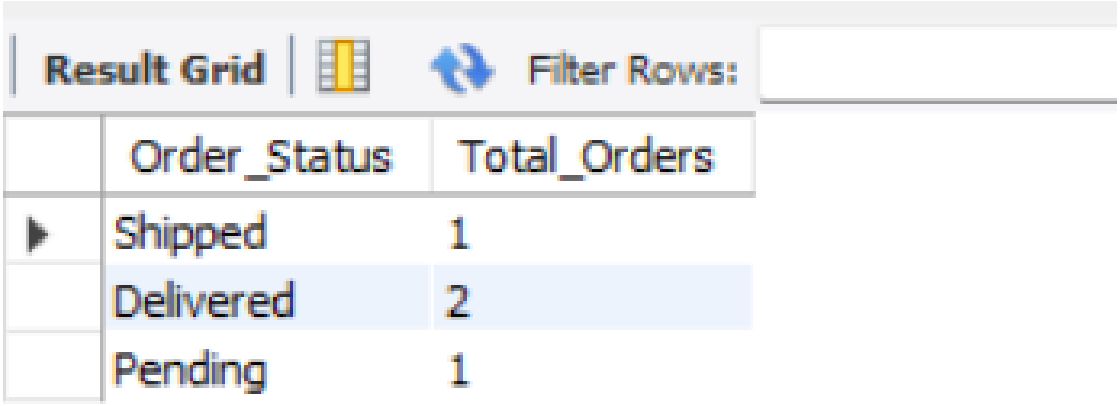
Result Grid | Filter Rows:

Product_Name	Total_Quantity_Sold
Laptop	1
Mouse	3
Keyboard	2
Headphone	1

Result Grid | Filter Rows:

Average_Order_Value
13875.000000





5. Result Summary

The database script creates the Customer, Product, Orders, and Order_Details tables, inserts sample data, and demonstrates data updates, deletion, joins, and summary queries. The screenshots document the outputs provided for the order and sales queries.

6. SQL Script

The original SQL script is included below for reference.

```
CREATE DATABASE OrderDB;
USE OrderDB;
CREATE TABLE Customer (
    Customer_ID INT PRIMARY KEY,
    Customer_Name VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    City VARCHAR(50)
);
CREATE TABLE Product (
    Product_ID INT PRIMARY KEY,
    Product_Name VARCHAR(50) NOT NULL,
    Price DECIMAL(10,2) NOT NULL CHECK (Price > 0)
);
```

```
CREATE TABLE Orders (
    Order_ID INT PRIMARY KEY,
    Customer_ID INT NOT NULL,
    Order_Date DATE NOT NULL,
    Total_Amount DECIMAL(10,2) DEFAULT 0
        CHECK (Total_Amount >= 0),
    Order_Status VARCHAR(20) DEFAULT 'Pending'
        CHECK (Order_Status IN
        ('Pending', 'Shipped', 'Delivered', 'Cancelled')),
    FOREIGN KEY (Customer_ID)
        REFERENCES Customer(Customer_ID)
);
CREATE TABLE Order_Details (
    Order_Detail_ID INT PRIMARY KEY,
    Order_ID INT NOT NULL,
    Product_ID INT NOT NULL,
    Quantity INT NOT NULL CHECK (Quantity > 0),
    Price DECIMAL(10,2) NOT NULL CHECK (Price > 0),
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),
    FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID),
    UNIQUE (Order_ID, Product_ID)
);
INSERT INTO Customer VALUES
(1, 'Aparna', 'aparna@gmail.com', 'Chennai'),
(2, 'Rahul', 'rahul@gmail.com', 'Delhi'),
(3, 'Priya', 'priya@gmail.com', 'Mumbai'),
(4, 'Kumar', 'kumar@gmail.com', 'Chennai');
INSERT INTO Product VALUES
(101, 'Laptop', 50000),
(102, 'Mouse', 500),
(103, 'Keyboard', 1000),
(104, 'Headphone', 1500);
INSERT INTO Orders VALUES
(1001, 1, '2026-10-20', 51000, 'Pending'),
(1002, 2, '2026-10-21', 2000, 'Shipped'),
(1003, 1, '2026-10-22', 1500, 'Delivered'),
(1004, 3, '2026-10-23', 500, 'Pending');
INSERT INTO Order_Details VALUES
(1, 1001, 101, 1, 50000),
(2, 1001, 102, 2, 500),
(3, 1002, 103, 2, 1000),
(4, 1003, 104, 1, 1500),
(5, 1004, 102, 1, 500);
SELECT * FROM Orders;
UPDATE Orders
SET Order_Status = 'Shipped'
WHERE Order_ID = 1001;
UPDATE Orders
SET Order_Status = 'Delivered'
WHERE Order_ID = 1002;
UPDATE Order_Details
SET Quantity = 3
WHERE Order_Detail_ID = 2;
UPDATE Orders
SET Total_Amount = (
    SELECT SUM(Quantity * Price)
    FROM Order_Details
    WHERE Order_ID = 1001
)
WHERE Order_ID = 1001;
DELETE FROM Order_Details
WHERE Order_ID = 1004;
DELETE FROM Orders
WHERE Order_ID = 1004
AND Order_Status = 'Cancelled';
SELECT
    o.Order_ID,
    c.Customer_Name,
    p.Product_Name,
```

```
    od.Quantity,
    od.Price,
    (od.Quantity * od.Price) AS Subtotal,
    o.Order_Date,
    o.Total_Amount,
    o.Order_Status
FROM Orders o
JOIN Customer c ON o.Customer_ID = c.Customer_ID
JOIN Order_Details od ON o.Order_ID = od.Order_ID
JOIN Product p ON od.Product_ID = p.Product_ID
ORDER BY o.Order_ID;
SELECT
    c.Customer_ID,
    c.Customer_Name,
    o.Order_ID,
    o.Order_Date,
    p.Product_Name,
    od.Quantity,
    od.Price,
    o.Order_Status
FROM Customer c
JOIN Orders o ON c.Customer_ID = o.Customer_ID
JOIN Order_Details od ON o.Order_ID = od.Order_ID
JOIN Product p ON od.Product_ID = p.Product_ID
ORDER BY c.Customer_ID, o.Order_Date;
SELECT
    p.Product_Name,
    o.Order_ID,
    od.Quantity,
    od.Price
FROM Product p
JOIN Order_Details od ON p.Product_ID = od.Product_ID
JOIN Orders o ON od.Order_ID = o.Order_ID;
SELECT
    p.Product_Name,
    COUNT(DISTINCT od.Order_ID) AS Times_Ordered
FROM Product p
LEFT JOIN Order_Details od ON p.Product_ID = od.Product_ID
GROUP BY p.Product_ID, p.Product_Name;
SELECT
    p.Product_Name,
    COALESCE(SUM(od.Quantity), 0) AS Total_Quantity_Sold
FROM Product p
LEFT JOIN Order_Details od ON p.Product_ID = od.Product_ID
GROUP BY p.Product_ID, p.Product_Name;
SELECT Customer_ID, Customer_Name, Order_Count
FROM (
    SELECT
        c.Customer_ID,
        c.Customer_Name,
        COUNT(o.Order_ID) AS Order_Count
    FROM Customer c
    JOIN Orders o ON c.Customer_ID = o.Customer_ID
    GROUP BY c.Customer_ID, c.Customer_Name
) AS T
WHERE Order_Count = (
    SELECT MAX(Order_Count)
    FROM (
        SELECT COUNT(*) AS Order_Count
        FROM Orders
        GROUP BY Customer_ID
    ) AS X
);
SELECT
    c.Customer_Name,
    COALESCE(SUM(o.Total_Amount), 0) AS Total_Spending
FROM Customer c
LEFT JOIN Orders o ON c.Customer_ID = o.Customer_ID
AND o.Order_Status <> 'Cancelled'
```

```
GROUP BY c.Customer_ID, c.Customer_Name;
SELECT AVG(Total_Amount) AS Average_Order_Value
FROM Orders
WHERE Order_Status <> 'Cancelled';
SELECT SUM(Total_Amount) AS Total_Sales
FROM Orders
WHERE Order_Status <> 'Cancelled';
SELECT Order_Status, COUNT(*) AS Total_Orders
FROM Orders
GROUP BY Order_Status;
```